订单生命周期与履约

3 态状态机 + RMQ/Cron 双保险 + 履约缺口路线图

gomall · 业务侧 deck

今日大纲

订单状态机的现状与差距

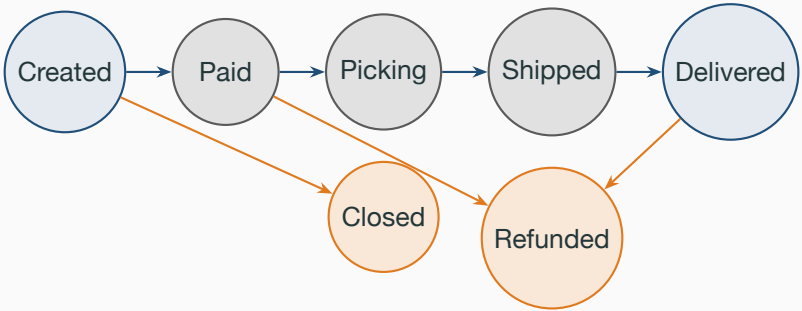
30 分钟关单 + RMQ/Cron 双保险

幂等关单与优雅降级

压测数字与履约缺口

订单状态机的现状与差距

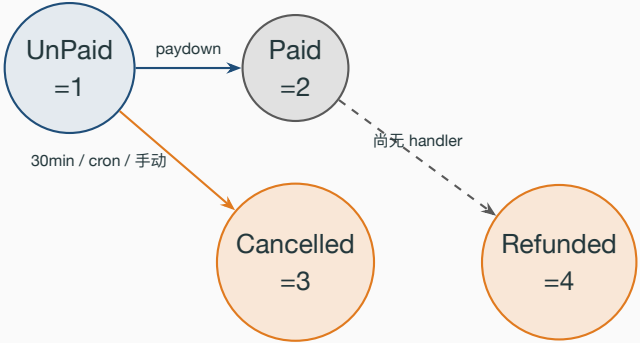
完整电商：7 态状态机



京东 / 拼多多 / 天猫

上线最少 7 个稳定状态，加上” 部分发货 / 部分退款” 扩展到 12–16 态。
真正复杂的不是状态本身，而是**状态之间的转移谁来推、靠什么消息触发、对账怎么做**。对照《电商系统设计》*(王新栋)* 第 *4* 章订单状态机；京东开放平台订单 *API* 状态枚举

gomall 现状：只有 3 态闭环



`consts/order.go:11--16` 定义了 UnPaid / Paid / Cancelled / Refunded 四个枚举。**有代码推动**的转移只有两条：UnPaid→Paid (paydown) 和 UnPaid→Cancelled (关单)。`Refunded` 是占位，目前没有任何接口能把订单写到这个状态。*consts/order.go:11–16* 状态枚举；*routes/router.go:114–121 paydown* 入口

两套常量并存的历史包袱

- `OrderTypeUnPaid=1, PendingShipping=2, Shipping=3, Receipt=4` ——老的”订单类型”。
- `UnPaid=1, Paid=2, Cancelled=3, Refunded=4` ——新的状态机。
- 两者数字 **1 重合 (=UnPaid)**、**2–4 各异**，DB 字段都叫 `type`。读老订单的代码会把 `Paid=2` 当成 `PendingShipping=2`，业务语义没冲突纯属巧合。
- 历史包袱必须靠一次性 **数据治理 PR** 来收口，靠新建状态机表 + 单向迁移。

consts/order.go:3–16 两组并列枚举

从”消息能不能送达”理解状态机难点

- Created→Paid：依赖**支付回调**。网络抖一下回调就丢，需要查询接口轮询。
- Paid→Picking：依赖 **WMS 拣货事件**。WMS 是另一套系统，得对账。
- Picking→Shipped：**物流接单接口**。顺丰 / 京东物流各一套 webhook。
- Shipped→Delivered：**快递公司签收回执**，延迟 0–72h，需 fallback 轮询。

核心难点**不是状态多**，而是每一次转移都跨系统、跨网络、跨组织边界。

对照美团技术博客 *2021*《订单状态机的演进》

30 分钟关单 + RMQ/Cron 双保险

30 分钟从哪定

方案	用户感受	业务后果
5 分钟	切卡 / 等短信都不够	商家投诉” 刚下单就被取消”，付款转化掉 10%+
15 分钟	多数人够用	高峰仍有 5%+ 误关，弱网用户最先被牺牲
30 分钟	含切卡 + 短信冗余	大促锁单可控，商家 / 运营都能接受
60 分钟	用户不抱怨	双 11 锁 1 小时，爆款一直显示” 已售罄”
24 小时	客服零投诉	库存周转崩盘，恶意囤货成本为零

商家投诉曲线和库存周转曲线是**反向的**，30 分钟是二者交点附近的工业经验值。

京东 30min / 天猫 30min–1h / 拼多多 15min ——越是高频低价类目越短。

*repository/rabbitmq/order_delay.go:21–22 OrderCancelDelay = 30 * time.Minute*

三处时长不一致 (一个待清理的债)

出现位置	数值	用途
`rabbitmq/order_delay.go:22`	**30 min**	TTL 延迟队列 (主)
`service/order_task.go:16`	**15 min**	Cron 兜底 (副)
`service/order.go:89`	**15 min**	旧 Redis ZSet (历史残留)

- Cron 用 15min 是**故意提前于 RMQ**：让兜底比主路径早动手，避免两路同时执行的并发去重压力。
- Redis ZSet 是早期实现，被 RMQ 替换后忘删，目前**只写不读**，等下一次清理 PR。
- 业务侧只关心 **用户实际看到 30 分钟**；技术侧三处数字的语义必须文档化。

repository/rabbitmq/order_delay.go:22　*service/order_task.go:16*　*service/order.go:89*

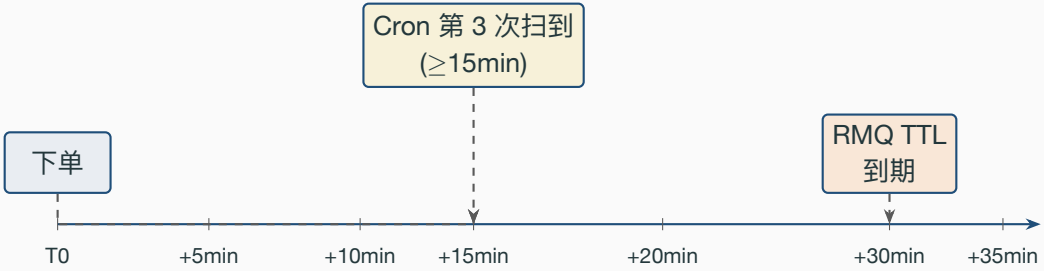
单一机制都不可靠

方案	致命弱点	何时露馅
只用 RMQ 延迟队列	MQ 宕机消息全丢	机房断网 / RabbitMQ 重启
只用 Cron 5min	高峰锁单累积 5–15min	双 11 库存周转跟不上
只用 Redis ZSet	Redis 内存满 evict	6M 历史订单一次性扫描

- RMQ 是**时效快**（30min 准点到秒级）、**失败盲**（消息丢就是丢）。
- Cron 是**时效粗**（5min 步进）、**保证强**（DB 全表扫一定能扫到）。
- 两个机制**失败模式互不相关**，组合后整体可靠性 $\approx 1 - p_{rmq} \cdot p_{cron}$。

cmd/main.go:54–62 tryInitRabbitMQ 设计意图

时间线：双保险怎么协同



- 正常路径：T+30min RMQ 投递死信 → `CancelUnpaidOrder` 关单。
- RMQ 失联：T+15min Cron 已关单 (`type=Cancelled`, RowsAffected=1)。
- RMQ 恢复后晚到的死信：`CloseOrderWithCheck` 因 `type≠UnPaid` no-op。

service/order_task.go:14–48 Cron；rabbitmq/order_delay.go:24–56 RMQ

RMQ 拓扑声明 (TTL + 死信)

Listing 1: *repository/rabbitmq/order_delay.go:34–55*

```
34 ch.ExchangeDeclare(orderDelayExchange, "direct", true, ...)
35 ch.ExchangeDeclare(orderDeadExchange, "direct", true, ...)
36
37 _, err = ch.QueueDeclare(orderDelayQueue, true, false, false, false,
38    amqp.Table{
39       "x-dead-letter-exchange": orderDeadExchange,
40       "x-dead-letter-routing-key": orderDeadRouting,
41    })
42 ch.QueueBind(orderDelayQueue, "", orderDelayExchange, false, nil)
43 ch.QueueDeclare(orderDeadQueue, true, false, false, false, nil)
44 ch.QueueBind(orderDeadQueue, orderDeadRouting, orderDeadExchange, ...)
```

L34–35: 两个 direct 交换机，一个收延迟消息、一个收死信。 L37–41: `x-dead-letter-exchange` 让 `delayQueue` 里 TTL 到期的消息自动转投死信交换机——TTL 延迟队列的标准做法。 L43–45: 消费者只盯 `deadQueue`，业务代码不直接接触 TTL 过程。

Cron 兜底主循环

Listing 2: *service/order_task.go:14–47*

```
14 func (s *OrderTaskService) RunOrderTimeoutCheck() {
15    orders, _ := baseDao.GetTimeoutOrders(15, 100)
16    for _, order := range orders {
17       baseDao.DB.Transaction(func(tx *gorm.DB) error {
18          ok, _ := txOrderDao.CloseOrderWithCheck(order.OrderNum)
19          if !ok { return errors.New("关单失败") }
20          ok, _ = txProductDao.RollbackStock(
21             order.ProductID, order.Num)
22          return nil
23       })
24    }
25 }
```

L16: `GetTimeoutOrders(15, 100)` 一次取 100 条，防 cron 单次跑过头。 L18: 每条订单独立事务，一条失败不连坐其他。 L20: `CloseOrderWithCheck` `WHERE type=UnPaid` 乐观锁，已被 RMQ 关掉者 `RowsAffected=0`。

真实 bug：cron 表达式打错 60 倍

Listing 3: *initialize/cron.go:16*

```
_, err := c.AddFunc("* */5 * * * *", func() { ... })
```

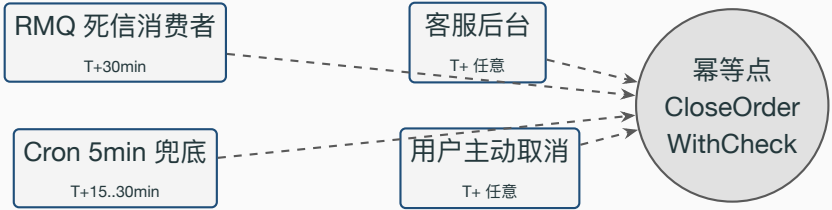
- `robfig/cron` 6 段格式：秒分时日月周。第一段 `*` 表示**每秒都跑**。
- 实际行为：**每 5 分钟里有 60 秒在跑**，触发 60 次。正确写法 `"0 */5 * * * *"`。
- 同文件 `cron.go:29` 红包任务写对了，可见复制改写时漏了一个 0。

cron 任务必须有**执行次数监控**：今天比昨天多跑 60 倍，告警立即响。

initialize/cron.go:16 vs cron.go:29 同文件对照

幂等关单与优雅降级

四个调用方都可能并发杀同一单



- RMQ + Cron 双保险已经天然导致**至少 2 个并发关单调用**。
- 客服处理工单时也可能手动取消；产品形态上未来还会加用户主动”取消订单”按钮。
- 任意一对调用并发命中，幂等点必须收敛——**有且只有一次**做”真关单 + 释放预占 + 发 outbox”的副作用。

service/order_cancel.go:14–63 幂等实现；rabbitmq/order_delay.go:96 RMQ requeue 一次

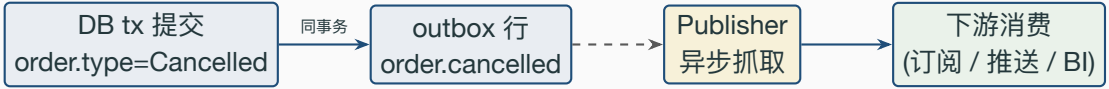
CancelUnpaidOrder 主体 (幂等 + Saga + outbox)

Listing 4: *service/order_cancel.go:19–63*

```go
func CancelUnpaidOrder(orderNum uint64) error {
   order, _ := baseDao.GetOrderByOrderNum(orderNum)
   if order == nil { return nil } // (1)
   var closed bool
   baseDao.DB.Transaction(func(tx *gorm.DB) error {
      ok, _ := dao.NewOrderDaoByDB(tx).CloseOrderWithCheck(orderNum) // (2)
      if !ok { return nil } // (3)
      closed = true
      return dao.NewOutboxDaoByDB(tx).Insert(...)
   })
   if closed { cache.ReleaseReservation(...) } // (4)
   return nil
}
```

(1) 不存在 no-op。 (2) `UPDATE WHERE type=UnPaid` 乐观锁。 (3) `RowsAffected=0` 时不发 outbox。 (4) Saga：库存退回 available。

为什么 outbox 必须在事务里



- 如果用 `tx.Commit()` → `MQ.Publish()` 两步，第二步失败会丢事件。
- Outbox 让事件**和业务数据同生死**：DB 提交即事件持久化，Publisher 异步搬运。
- 这就是 `order_cancel.go:40--50` 把 `OutboxDao.Insert` 放在 `Transaction` 回调里的原因。

service/order_cancel.go:40–50；配套 deck 04-outbox-saga.tex

已知缺陷：orders/delete 旁路

Listing 5: *routes/router.go:99 + service/order.go:185*

```
authed.POST("orders/delete", api.DeleteOrderHandler())
// service/order.go:185
err = db.DeleteOrderById(req.OrderId, u.Id) // 物理删除，无任何检查
```

- 当前实现：**物理删除**订单行，不走 `CancelUnpaidOrder`。
- 后果一：**库存泄漏**——Redis `reserved` 没释放。
- 后果二：**outbox 漏写**——下游 BI / 退款看不到这单。
- 后果三：**资金已扣**的 Paid 单也能被删，账实分离。

修复：`DeleteOrder` 内部转调 `CancelUnpaidOrder`；对 Paid 单返 4xx 引导走退款。

routes/router.go:99 + service/order.go:185 缺补偿

tryInitRabbitMQ 静默降级

Listing 6: *cmd/main.go:54–62*

```go
54 func tryInitRabbitMQ() {
55    defer func() {
56       if r := recover(); r != nil {
57          util.LogrusObj.Warnf("RMQ init 失败: %v", r)
58       }
59    }()
60    rabbitmq.InitRabbitMQ()
61    initialize.InitOrderDelayConsumer()
62 }
```

启动顺序 `InitCron()` → `tryInitRabbitMQ()`：**Cron 先就位**。RMQ 连不上 → recover 吞 panic → HTTP 服务正常起，30min 主路径暂停但 15min Cron 照常每 5min 扫一次。用户感受：关单时间从 30min 漂到 15–20min，提前关不算 bug。*cmd/main.go:35–37 启动顺序确保 Cron 在 RMQ 之前*

两套基础设施挂掉后的影响矩阵

RMQ	Cron	用户感受	数据风险
在	在	30min 准点关单	无
挂	在	15–20min 提前关	无 (提前优于漏关)
在	**挂**	30min 准点关单	RMQ 单点风险
挂	**挂**	**永不关单**	库存泄漏，需手动 reconcile

- 两套同时挂，需要值班同学跑 reconcile 脚本 (按 `created_at` 扫 `type=UnPaid`)。
- 这是为什么 `ReserveStock` 必须**独立于关单链路**有 TTL 兜底 (deck 11 业务侧库存)。

repository/cache/inventory.go reserved 字段；deck 11-inventory-business.tex 配套

压测数字与履约缺口

REPORT.md 里关于订单的关键锚点

场景	RPS	p95	备注
/orders/list (游标 + 缓存)	**58,406**	5.0ms	PR #38 优化
/orders/list (旧版深分页)	8.3	15,950ms	反面教材
/orders/create (幂等重放)	50,000+	2.3ms	同 Key 重放命中 done 态

- 同 `Idempotency-Key` 15s 打 **755,033 次** `/orders/create`，DB 只生成 **1 笔订单**。
- 同一把幂等锁也是关单链路的并发模型：RMQ + Cron + 客服三路同时打 `CloseOrderWithCheck` 不会重复释放库存。
- 真正瓶颈在**下游支付 / 物流 / 退款的系统边界**。

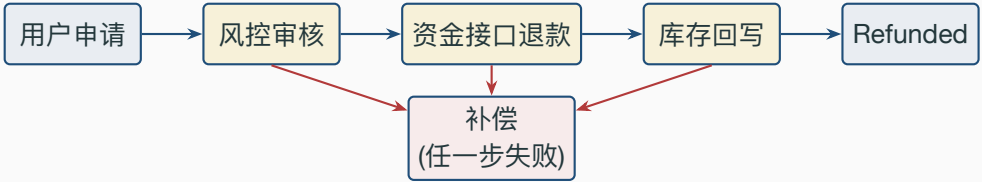
stressTest/REPORT.md § 链路压测 + 幂等重放

从 Paid 之后 gomall 没有任何 handler

- `paydown` handler 写 `type=Paid` 后，订单数据**再也没人动**。
- 真实电商需要：拣货 / 出库 / 物流单 / 签收 / 售后五条独立链路。
- 每条链路都是**跨系统集成**——不是写代码的工作量，是**对接 API 和签合同**的工作量。

routes/router.go:114–121 paydown 是当前业务链路终点

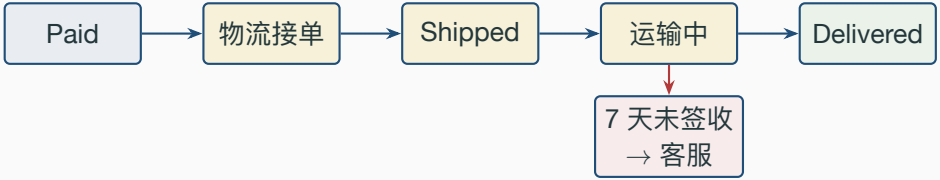
退款 Saga 路线图：理论形态



- 5 步链路里只要任意一步失败，前面已成功的步骤都要走**补偿动作**。
- 资金接口退款 (第三方支付 API) 是**慢且不可控**的依赖——需要熔断 (deck 05) + outbox (deck 04) 兜底。
- gomall 现状：枚举值 `Refunded=4` 已经留位，handler / DAO / 第三方支付 SDK 都没接。

consts/order.go:15 Refunded 枚举占位；deck 04-outbox-saga.tex Saga 模式

物流 + 签收链路：webhook + 轮询双保险



- 物流接单：调顺丰 / 京东物流下单 API，拿到运单号写回订单。
- Shipped 推送：物流公司 webhook 推 **揽件成功** → 业务侧改状态。
- Delivered 兜底：webhook 不可信，每天**轮询查件**对账。
- gomall 现状：DB 里有 `Shipping=3` / `Receipt=4` 的老枚举但没人写。

consts/order.go:6–7 老枚举占位

售后链路：远比退款复杂

售后类型	业务复杂度
仅退款 (Refund Only)	钱回去就行，库存不动
退货退款 (Return + Refund)	用户寄回 + 入库质检 + 退钱 + 库存加回
换货 (Exchange)	退货 + 新发货，资金不动
价保 (Price Match)	不退货只退差额，营销系统介入
仅维修 (Repair Only)	寄回 + 维修 + 寄回，跨 30–90 天

- 五种售后**业务流程完全不同**，但都要在订单上记录”已发起售后单”状态。
- 大型电商通常用**售后单**另一张表，订单只标”有 N 个售后单关联”。
- gomall 当前没有任何售后表——第一版能接通”仅退款”就是巨大进步。

对照淘宝开放平台《退款 / 退货接口》文档结构

面试 Q&A

Q1：为什么不只用 RMQ，要再加 Cron?

A：RMQ 宕机或消息丢就关不了单。Cron 在 DB 扫，失败模式互不相关，整体可用率 $\approx 1 - p_{rmq} \cdot p_{cron}$。

Q2：`CancelUnpaidOrder` 怎么保证幂等?

A：`UPDATE WHERE type=UnPaid` 乐观锁，`RowsAffected=0` 时 no-op，不释放库存不发 outbox。四调用方共用这把锁。

Q3："`* */5 * * * *`" 为什么是 bug?

A：6 段格式第一段是秒，`*` 每秒都跑。在分钟% 5==0 时 5min 跑 60 次。正确 "`0 */5 * * * *`"。

Q4：状态机为什么只有 3 态，缺了什么?

A：缺 Picking / Shipped / Delivered / Refunded ——每态都要跨系统对接 WMS / 物流 / 快递 / 资金接口。

代码位置一览

Cron 兜底 *service/order_task.go:14–48*

幂等关单 +Saga+outbox *service/order_cancel.go:14–63*

乐观锁 UPDATE *repository/db/dao/order.go:185–195*

RMQ 拓扑 + 发送 + 消费 *repository/rabbitmq/order_delay.go:24–105*

双保险注册 *initialize/cron.go:13–53*

静默降级 + 启动顺序 *cmd/main.go:35–37, 54–62*

状态枚举两组 *consts/order.go:3–16*

orders/delete 缺陷 *routes/router.go:99 + service/order.go:185*

配套 deck：15-order-close-business / 04-outbox-saga / 11-inventory-business。